

National College of Ireland
Higher Diploma in Science in Computing
HDCSDEV_INT

Terminal Assignment Based Assessment (TABA) – May 2025
Distributed Systems

Student Name: Guilherme Silveira

Student ID: 23413271

Lecturer: Catriona Nic Lughadha

Submission Date: 24 May 2025

Question 1

a) What are the primary differences between REST and gRPC in terms of performance and communication styles?

REST is an architectural style (not a framework) based on standard HTTP methods (GET, POST, PUT and DELETE) that usually transmits data in human-readable formats such as JSON. Because it relies on text-based communication and stateless interactions, it tends to be slower and less efficient for large or complex data transfers.

On the other hand, gRPC is a framework that uses HTTP/2 for transport and Protocol Buffers, a compact binary format, for data serialization. Therefore, this results in much faster communication and reduced payload size. gRPC supports multiple communication styles, including unary calls, client streaming, server streaming, and bidirectional streaming.

In conclusion, even though REST is more human-friendly and widely adopted, gRPC offers better performance and richer communication patterns. While REST can be implemented through many frameworks (e.g., Spring Boot), gRPC itself is a full-featured framework for building efficient remote procedure call systems.

(GeeksForGeeks, 2024)

b) Describe Peer-to-Peer (P2P) systems and mention some real-world examples.

A Peer-to-Peer (P2P) system is a network model where each device, called a “peer”, can act both as a client or a server, which means that, instead of relying on a central server, peers share resources directly with each other. Therefore, there is no central point of control, each peer is responsible for contributing and requesting services, and resources like files, processing power, or bandwidth are distributed among the peers. This makes it a decentralized and resilient system, since there is no single point of failure.

Some real-world examples are:

- **BitTorrent:** A file-sharing protocol where users download and upload parts of files to each other instead of a central server.
- **Skype (original architecture):** Used P2P model to route voice data between users.
- **Blockchain / Bitcoin:** Each node (peer) maintains a full copy of the ledger and validates transactions collaboratively.

(GeeksForGeeks, 2024) (BitTorrent, 2025) (Rickard, 2022) (Bitcoin, 2025)

c) What is the role of a message broker in a publish-subscribe system?

In a publish-subscribe system, a message broker is a middle component that helps send messages from one system (the publisher) to many receivers (the subscribers), without them having to know about each other. Examples of brokers include RabbitMQ and Apache Kafka.

Instead of a publisher directly sending messages to each subscriber, the message is sent to the broker, and the broker handles the rest. It receives messages from publishers, stores or routes them, and delivers them to subscribers who are interested in that topic or type of message.

(GeeksForGeeks, 2024)

d) What is service discovery in microservices and how does it work in distributed environments?

In a microservice architecture, service discovery is the mechanism that allows services to automatically find and connect to each other in a dynamic and distributed environment.

Since services are often deployed in containers or cloud environments, their locations (like IP addresses or ports) can change frequently. Rather than hardcoding these addresses, service discovery handles the automatic registration and lookup of services. To do that, there are usually two main components:

1. Service Registry – A database where services register themselves, providing their locations (IP addresses and ports)
2. Service Discovery Client – When one service needs to talk to another, it queries the registry to find out where the target service is currently running.

(Rahman, 2024)

e) How does UDP (User Datagram protocol) ensure fast communication despite not guaranteeing delivery?

UDP (User Datagram Protocol) is a lightweight protocol used for fast communication in networks. It sends messages (datagrams) without setting up a dedicated connection, which is what makes it fast, but unreliable. There are no handshakes, acknowledgments or re-transmissions, and messages are just sent out and have less metadata (smaller headers). Therefore, data can be sent very quickly.

It is used when speed matters more than reliability, or when the application can handle lost data itself. A real-world example of it is Video Streaming, since it is ok to lose a few frames to avoid buffering.

(GeeksForGeeks, 2024)

f) What are the core challenges in building scalable distributed systems?

Some core challenges developers face when building scalable distributed systems are:

1. **Data consistency:** Data must be synchronized in real time across nodes.
2. **Network Latency and Communication Overhead:** Nodes may be geographically far apart, which leads to delays that can affect performance
3. **Fault Tolerance:** Systems must keep running even when parts fail. This requires redundancy, failover mechanisms, and handling of partial failures
4. **Load Balancing:** Workload should be distributed evenly. Some nodes may become bottlenecks without proper balancing.
5. **Service Discovery and Dynamic Scaling:** Services need to find and communicate with each other even as they scale up and down.
6. **Security and Access Control:** More components mean more potential attack points. Ensuring secure communication, authentication, and authorization becomes critical.

These challenges must be considered from the beginning of system design, or the system may become unreliable, slow, or unable to grow efficiently.

(GeeksForGeeks, 2024)

Question 2

This task involved building a distributed system in Java using gRPC. The system allows a client to send an array of integers to a server, which calculates the sum and returns it to the client.

System Components:

sum.proto

Defined the `SumService` with a `SumArray` method. The `SumRequest` holds an array of integers, and `SumResponse` returns the sum.

```
1  syntax = "proto3";
2
3  option java_multiple_files = true;
4  option java_package = "com.sumservice";
5  option java_outer_classname = "SumServiceProto";
6
7  service SumService {
8      rpc SumArray (SumRequest) returns (SumResponse);
9  }
10
11  message SumRequest {
12      repeated int32 numbers = 1;
13  }
14
15  message SumResponse {
16      int32 result = 1;
17  }
```

SumServer.java

- Starts a gRPC server on port 50051
- Implements the service logic to sum integers
- Sends the result back using StreamObserver

```

14 public class SumServer {
15     public static void main(String [] args) throws IOException, InterruptedException {
16         // Create and start the server on port 50051
17         Server server = ServerBuilder.forPort(50051) // Starts gRPC server on port 50051
18             .addService(new SumServiceImpl()) // links the generated stub class to implemented logic
19             .build();
20
21         System.out.println("Server started on port 50051");
22         server.start();
23         server.awaitTermination(); // keep the server running
24     }
25
26     // Inner class to handle the service
27     static class SumServiceImpl extends SumServiceGrpc.SumServiceImplBase {
28         @Override //Overrides sumArray() method generated from the proto file
29         public void sumArray(SumRequest request, StreamObserver<SumResponse> responseObserver) {
30             int sum = 0;
31
32             // Sum all number from the request
33             for (int number : request.getNumbersList()) {
34                 sum += number;
35             }
36
37             // Create the response
38             SumResponse response = SumResponse.newBuilder().setResult(sum).build();
39
40             // Send the response
41             responseObserver.onNext(response);
42             responseObserver.onCompleted();
43         }
44     }
45 }

```

SumClient.java

- Connects to the server
- Sends a request with a list of integers
- Receives and displays the result using a blocking stub

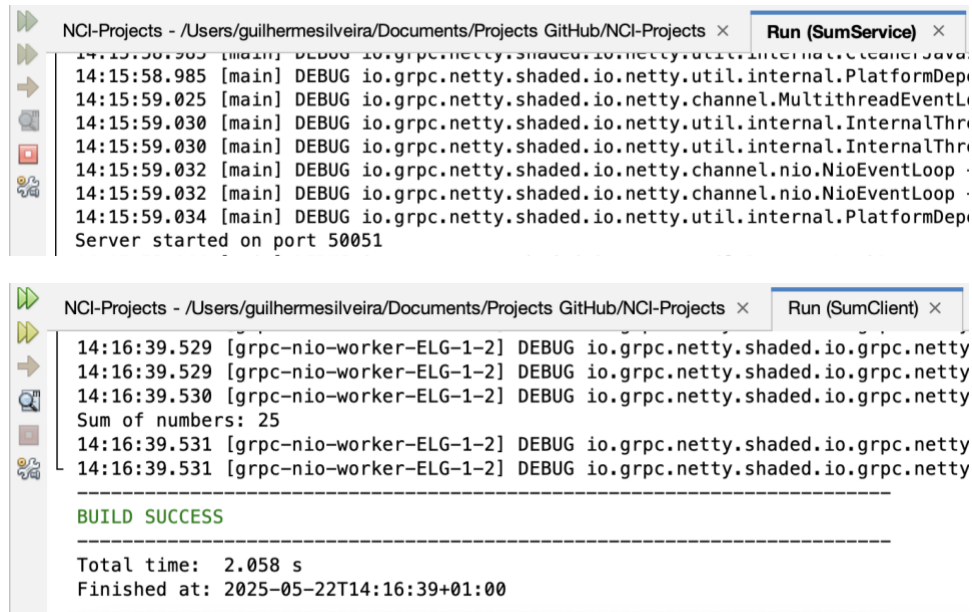
```

14 public class SumClient {
15     public static void main(String[] args){
16         // Open a channel to the server
17         // ManagedChannelBuilder connects client to gRPC server
18         ManagedChannel channel = ManagedChannelBuilder.forAddress("localhost", 50051)
19             .usePlaintext() // disables TLS
20             .build();
21
22         // Create a stub from the generated gRPC class
23         SumServiceGrpc.SumServiceBlockingStub stub = SumServiceGrpc.newBlockingStub(channel);
24
25         // Prepare a list of numbers to sum
26         List<Integer> numbers = Arrays.asList(3, 5, 7, 10);
27
28         // Create a request
29         SumRequest request = SumRequest.newBuilder()
30             .addAllNumbers(numbers)
31             .build();
32
33         // Send request and receive response
34         SumResponse response = stub.sumArray(request);
35
36         // Print result
37         System.out.println("Sum of numbers: " + response.getResult());
38
39         // Shutdown channel
40         channel.shutdown();
41     }
42 }

```

Input / Output

- Client sends: `List<Integer> numbers = Arrays.asList(3, 5, 7, 10);`
- Output (Server response): Sum of numbers: 25



```

NCI-Projects - /Users/guilhermesilveira/Documents/Projects GitHub/NCI-Projects x Run (SumService) x
14:15:58.985 [main] DEBUG io.grpc.netty.shaded.io.netty.util.internal.CleanerJava
14:15:58.985 [main] DEBUG io.grpc.netty.shaded.io.netty.util.internal.PlatformDep
14:15:59.025 [main] DEBUG io.grpc.netty.shaded.io.netty.channel.MultithreadEventL
14:15:59.030 [main] DEBUG io.grpc.netty.shaded.io.netty.util.internal.InternalThre
14:15:59.030 [main] DEBUG io.grpc.netty.shaded.io.netty.util.internal.InternalThre
14:15:59.032 [main] DEBUG io.grpc.netty.shaded.io.netty.channel.nio.NioEventLoop
14:15:59.032 [main] DEBUG io.grpc.netty.shaded.io.netty.channel.nio.NioEventLoop
14:15:59.034 [main] DEBUG io.grpc.netty.shaded.io.netty.util.internal.PlatformDep
Server started on port 50051

NCI-Projects - /Users/guilhermesilveira/Documents/Projects GitHub/NCI-Projects x Run (SumClient) x
14:16:39.529 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty
14:16:39.529 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty
14:16:39.530 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty
Sum of numbers: 25
14:16:39.531 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty
14:16:39.531 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty

BUILD SUCCESS

Total time: 2.058 s
Finished at: 2025-05-22T14:16:39+01:00
  
```

Synchronous vs Asynchronous RPC in gRPC

In gRPC, a client can interact with the server using either synchronous or asynchronous calls, depending on how the request is made and how the response is handled.

In a synchronous call, the client sends a request and then waits until the server responds. While waiting, the client cannot do anything else, it is blocked. This is the type of call used in this implementation, where the client sends the array, and the server responds with the sum before the client continues. This approach is simple and works well for small tasks where the response is quick, like summing numbers.

On the other hand, in an asynchronous call, the client sends a request but does not wait. It keeps running and handles the server's response later, usually through a callback. This is helpful when the server might take a while to respond, or when the client needs to stay responsive. This approach is more flexible, but a bit more complex to manage.

The choice between synchronous and asynchronous RPCs affects how the client and server interact. With synchronous RPC, the client is tightly coupled to the server's response time – the client must wait, which can slow things down if the server is busy. In contrast, asynchronous RPC allows the client to stay responsive and perform other tasks while waiting for the server's reply, making the overall system more efficient and scalable, especially in environments with high load or variable response times.

(GeeksForGeeks, 2024) (gRPC, 2025)

Question 3

A distributed system is made up of different computers (or nodes) that are connected and work together to provide a single, unified system to the user. Even though these nodes may be in different places, they collaborate to perform tasks as if they were part of one machine.

Some fundamental characteristics of distributed systems are:

1. **Resource sharing:** The whole point of a distributed system is to share resources – whether it is storage, data, processing power, or services – across different machines.
2. **Concurrency:** Multiple things happen at the same time. Since different parts of the system are running in parallel, they may compete for resources or communicate with each other constantly.
3. **Transparency:** One of the goals of distributed systems is to make the complexity invisible. Users do not need to know where the resources are located or how many nodes are involved – everything should feel seamless.
4. **Scalability:** Distributed systems should be able to grow easily. Whether there are 10 or 10,000 users, the system should continue working smoothly.
5. **Fault Tolerance:** If something goes wrong – for example, server crashes – the system should still work. It needs to detect the problem and recovers automatically, without stopping the whole service.

Some of the main challenges that affect distributed systems are:

Network Latency

In distributed systems, nodes have to communicate over a network and that takes time. This delay (even if it is small) can affect performance and the user experience. To handle this, systems must be built to deal with slow responses or temporary disconnects using retries, timeouts, or fallbacks.

Scalability

As the number of users or requests grows, the system must scale up without becoming unstable. This usually means adding more machines or services – which sounds easy but only works if the system was designed with scalability in mind. For example, services must be stateless so any machine can handle any request, and data may need to be split or duplicated across services.

Fault Tolerance

Since distributed systems involve many components, some of them will eventually fail. A good system should continue running even if one or more parts stop working. This is done using redundancy (having backups), detecting failures quickly (with heartbeat signals), and recovering or switching to another service when needed.

Distributed systems allow us to combine many machines into one powerful system, but they are not easy to manage. Challenges like latency, scalability, and failure require careful planning and good architecture. If these things are not considered, the system may become slow, unreliable, or unable to grow as needed.

(Naveenjain, 2024) (GeeksForGeeks, 2025) (GeeksForGeeks, 2024)

Question 4.1

In distributed systems, the components are spread across multiple networks and devices, which opens up more opportunities for security risks. Unlike a traditional centralized system, a distributed setup has many points where an attacker could try to access, intercept, or modify data.

Some common security risks are:

1. **Eavesdropping (Data interception):** Attackers can listen to messages as they travel between services. This is especially risky if the data includes login info, tokens, or private messages.
2. **Man-in-the-middle Attacks (MITM):** An attacker could pretend to be a legitimate part of the system to intercept or change messages before they reach their real destination.
3. **Unauthorized Access:** If access control is weak, users or attackers might access services they are not supposed to – like seeing private data or performing restricted actions.
4. **Data Tampering:** Without proper protection, data can be modified in transit, which may lead to corrupted operations or misleading results.
5. **Replay Attacks:** An attacker captures a valid request and sends it again later to perform the same action (e.g., re-sending a payment request).

Authentication and Authorization

- **Authentication** is the process of checking if the user (or service) is who they claim to be. For example: login systems, API tokens, or digital certificates.
- **Authorization** happens after authentication – it decides what the user or service is allowed to do. For example: a regular user can view data, but an admin can delete it.

In distributed systems, both processes happen at every service boundary, since different services may have different roles, permissions, or owners.

Security Protocols Commonly Used

1. **TLS/SSL (Transport Layer Security):** Encrypts data in transit to prevent eavesdropping or tampering.
2. **HTTPS (HTTP over TLS):** Secure version of HTTP – standard for secure web and API communication.

3. **OAuth 2.0:** Used for authorization – lets apps access user data securely (used by Google, Facebook logins).
4. **JWT (JSON Web Tokens):** A compact token format used to pass identity and authorization claims between services. It is often used in stateless microservices systems.
5. **mTLS (Mutual TLS):** Adds client authentication to TLS so both the client and server verify each other – great for internal service-to-service security.

In summary, distributed systems suffer more security risks because of their complexity and wide surface area. Authentication and authorization must be handled carefully, ideally at every service boundary. Security protocols like TLS, JWT, and OAuth help protect data, verify identity, and control access.

(GeeksForGeeks, 2024) (Delord, 2024) (Okta Developer, 2025) (GeeksForGeeks, 2024)

Question 4.3

The CAP Theorem is an important concept in distributed systems that helps us understand the limitations of what a system can guarantee when things go wrong – especially during a network failure.

CAP stands for: **C**onsistency, **A**vailability, and **P**artition Tolerance

The theorem says that in a distributed system, you can only guarantee two out of these three at the same time – but not all three fully, as illustrated in the figure below:

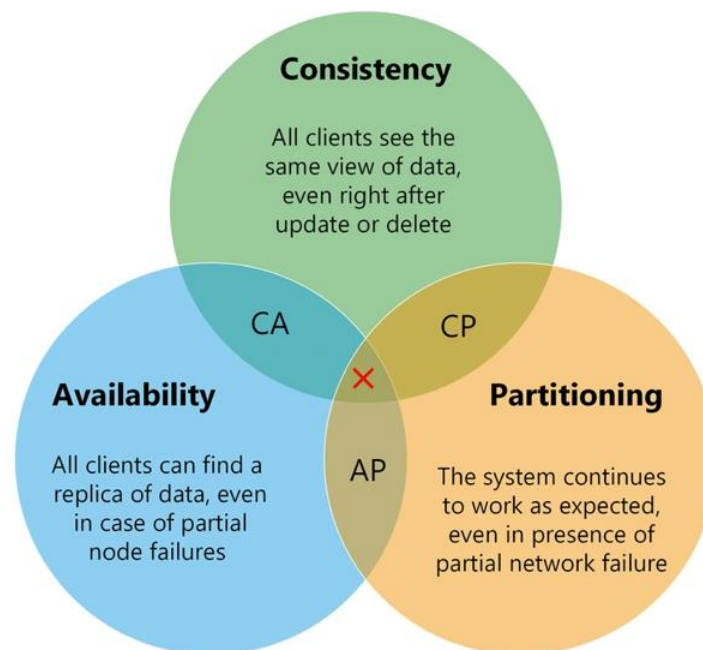


Figure 1: CAP Theorem (Cherif, 2024)

1. Consistency

This means that all nodes in the system see the same data at the same time. If I write something to one server, and you read from another, you should see exactly what I just wrote.

Example: In a banking system, if I transfer money, the balance should be updated instantly everywhere.

2. Availability

Availability means that the system responds to requests – even if it is not the most up-to-date data. Users should always get a response, even during network issues or high load.

Example: When you are browsing a shopping site during Black Friday and it does not crash, even if some numbers (like stock) are not perfectly updated.

3. Partition Tolerance

This means the system continues to work even when there is a network failure between parts of the system (called a “partition”). In real life, network issues are almost guaranteed, so partition tolerance is usually a must.

Example: If a system has nodes in Ireland and the US, and the network between them breaks, they should still work – even if some data syncing is delayed.

The Trade-Offs

The key idea in CAP is that you should choose between consistency and availability when there is a network partition:

- A **CP (Consistency + Partition Tolerance)** system prefers correct data, but might reject some requests during failure to avoid serving outdated data.
Example: MongoDB in “strict” mode, or some bank systems.
- An **AP (Availability + Partition Tolerance)** system will always respond, even if the data is not totally up to date.
Example: DNS servers or shopping carts that eventually sync.
- A **CA (Consistency + Availability)** system is only possible if there are no network partitions, which is unrealistic in distributed systems.

The CAP Theorem helps us understand the limits of distributed systems. We cannot have consistency, availability and partition all at once. So, system designers must choose the trade-off that makes sense for their use case – often giving up a bit of consistency or availability depending on what is more important.

(IBM, 2022) (Cherif, 2024)

References

Bitcoin, 2025. *Bitcoin - Open source P2P money*. [Online]

Available at: <https://bitcoin.org/en/>

[Accessed 20 May 2025].

BitTorrent, 2025. *About bitTorrent*. [Online]

Available at: <https://www.bittorrent.com/company/about-us/>

[Accessed 20 May 2025].

Cherif, Y., 2024. *Understanding the CAP Theorem in Distributed Systems*. [Online]

Available at: https://dev.to/yasmine_ddec94f4d4/understanding-the-cap-theorem-in-distributed-systems-5230

[Accessed 22 May 2025].

Delord, S., 2024. *Authentication and Authorization in Red Hat OpenShift and Microservices Architectures*. [Online]

Available at: <https://www.redhat.com/en/blog/authentication-authorization-redhat-openshift-microservices-architectures>

[Accessed May 2025].

GeeksForGeeks, 2024. *Common Problems in Distributed Systems and their Solutions*.

[Online]

Available at: <https://www.geeksforgeeks.org/common-problems-in-distributed-systems-and-their-solutions/>

[Accessed 22 May 2025].

GeeksForGeeks, 2024. *Difference Between Synchronous and Asynchronous Transmission*.

[Online]

Available at: <https://www.geeksforgeeks.org/difference-between-synchronous-and-asynchronous-transmission/>

[Accessed 22 May 2025].

GeeksForGeeks, 2024. *gRPC vs. REST: Top Differences*. [Online]

Available at: <https://www.geeksforgeeks.org/grpc-vs-rest/>

[Accessed 20 May 2025].

GeeksForGeeks, 2024. *Peer-to-Peer (P2P) Architecture*. [Online]

Available at: <https://www.geeksforgeeks.org/peer-to-peer-p2p-architecture/#p2p-applications-use-cases>

[Accessed 20 May 2025].

GeeksForGeeks, 2024. *Scaling Distributed Systems*. [Online]
Available at: <https://www.geeksforgeeks.org/scaling-distributed-systems/>
[Accessed 20 May 2025].

GeeksForGeeks, 2024. *Security in Distributed System*. [Online]
Available at: <https://www.geeksforgeeks.org/security-in-distributed-system/>
[Accessed 22 May 2025].

GeeksForGeeks, 2024. *User Datagram Protocol (UDP)*. [Online]
Available at: <https://www.geeksforgeeks.org/user-datagram-protocol-udp/>
[Accessed 20 May 2025].

GeeksForGeeks, 2024. *What are Message Brokers in System Design?*. [Online]
Available at: <https://www.geeksforgeeks.org/what-are-message-brokers-in-system-design/>
[Accessed 20 May 2025].

GeeksForGeeks, 2025. *Design Issues of Distributed System*. [Online]
Available at: <https://www.geeksforgeeks.org/design-issues-of-distributed-system/>
[Accessed 22 May 2025].

gRPC, 2025. *Basics tutorial*. [Online]
Available at: <https://grpc.io/docs/languages/java/basics/>
[Accessed 23 May 2025].

IBM, 2022. *What is the CAP Theorem?*. [Online]
Available at: <https://www.ibm.com/think/topics/cap-theorem>
[Accessed 12 May 2025].

Naveenjain, 2024. *Distributed Systems: Basics*. [Online]
Available at: <https://medium.com/@naveenjain213.nj/distributed-systems-basics-78292e1e437e>
[Accessed May 2025].

Okta Developer, 2025. *OAuth 2.0 and OpenID Connect overview*. [Online]
Available at: <https://developer.okta.com/docs/concepts/oauth-openid/>
[Accessed 22 May 2025].

Rahman, J., 2024. *Understanding Service Discovery in Microservices Architecture*. [Online]
Available at: <https://medium.com/@jeslurrahman/understanding-service-discovery-in-microservices-architecture-2098b10e7439>
[Accessed May 2025].

Rickard, M., 2022. *The Centralization of Skype*. [Online]
Available at: <https://mattrickard.com/the-centralization-of-skype>
[Accessed 20 May 2025].